S.A.N.T.A. x OmegaClaw — Sprint Report

BGI Nexus Hackathon · June 28, 2026 · Kristian Dansen

What we set out to do

The mission was simple to state and hard to execute: build an autonomous agent that discovers trustworthy local service providers not by scraping reviews, but by reasoning from independently verifiable evidence. We called it S.A.N.T.A. — System for Authenticating Neighborhood Trust and Accountability.

The core hypothesis was that OmegaClaw, with its persistent memory and agentic architecture, could function as a scientific researcher rather than a search engine. Instead of asking "which company looks trustworthy?", SANTA would ask "which company has demonstrated trustworthy behaviour through evidence that cannot be faked?" KvK registration, trade certifications, physical address verification, community participation — these are things that take years to build and cannot be purchased overnight.

The intended output was a structured, open-source database of verified local contractors, each with a multi-dimensional Trust Vector instead of a single opaque score. We chose painting contractors in the Lisse/Leiden region of the Netherlands as our pilot sector and geography.

What we actually built

We produced five design documents defining the SANTA architecture: a research cycle methodology, a trust dimensions framework, a source trust scoring model, a negative signals registry for manipulation detection, and an output structure specification. These are real conceptual contributions. The architecture they describe is sound.

We ran OmegaClaw in a live Docker environment via Telegram, across a full production sprint. The agent actively searched, queried, and attempted to compile a database of 26 painting companies across six municipalities. It cross-referenced sources including the KvK Handelsregister, Ondernemend Lisse, Trustoo, and Vebidak. It hit eight distinct technical walls and resolved most of them. It generated a companies.json file with structured records containing names, addresses, KvK numbers, and trust scores.

And then we discovered the critical failure.

The failure we must be honest about

The KvK data in companies.json is hallucinated. Every record.

At some point during the sprint, OmegaClaw encountered a series of cascading failures: permission errors that blocked writes, a context window that flooded past 61,000 tokens, a search API that silently returned nothing for Dutch business queries. Real data was retrieved during earlier search cycles but never persisted to disk correctly. When the agent eventually generated the output files, it had lost access to the real data it had found. Rather than stopping and flagging this uncertainty, it filled the gaps with plausible-sounding fabrications. KvK numbers, addresses, phone numbers, email addresses — all invented with high syntactic confidence and zero factual basis.

The trust scores attached to these hallucinated records range from 0.78 to 0.91. They look authoritative. They are completely meaningless.

This is not a minor quality issue. For a project whose entire mission is verifiable truth, outputting fabricated registry data is a fundamental contradiction. We are naming it clearly because the BGI community deserves honesty, because the judges deserve honesty, and because identifying the failure precisely is the only way to fix it.

What we learned about why this happened

The failure reveals a structural problem in how OmegaClaw handles the boundary between retrieved data and generated text.

When a human researcher takes notes, there is a clear cognitive distinction between "I wrote this down from a source" and "I am now filling in a gap from memory or inference." OmegaClaw does not currently maintain this distinction in a way that survives context pressure. When the context window flooded, when writes failed silently, when the search tool returned nothing — the agent continued producing output without flagging that the evidential basis for that output had collapsed. It behaved as if continuing was better than stopping.

This points to three specific architectural gaps.

The first is the absence of an evidence provenance chain. Every data point needs to carry a tag identifying exactly where it came from — a specific URL, a specific tool call, a specific timestamp — and this tag needs to be written to persistent storage at the moment of retrieval, not reconstructed later from context. If the provenance tag cannot be attached, the data point should not be written.

The second is the lack of a write-before-discard discipline. The agent accumulated real findings in its context window but treated that window as reliable working memory. It is not. The context window is ephemeral and lossy under pressure. Every confirmed fact needs to be committed to disk the moment it is verified, not batched for later.

The third is the absence of an uncertainty halt. When confidence in the evidential basis drops below a threshold — because sources are unreachable, because verification fails, because the agent has been searching the same query repeatedly without new results — the correct behaviour is to stop and report the uncertainty, not to complete the task by

inference. The agent currently has no mechanism for this. It optimises for task completion over epistemic honesty.

What the sprint proved despite the failure

The hallucination failure does not invalidate everything. Several things were genuinely demonstrated.

OmegaClaw proved it can adapt autonomously when tools fail. When the Tavily search API returned nothing, the agent diagnosed its own environment, identified that Python urllib was available, wrote a custom HTTP scraping script, and successfully retrieved real HTML search results from Google. It found actual painting companies in the Lisse and Leiden area through this self-engineered workaround. That data was real before it was lost. The capacity for autonomous adaptation is genuine.

The dual-memory architecture — ChromaDB for semantic vector search, MeTTa for chronological symbolic history — successfully maintained geographic context across a long production session. The agent remembered the research parameters, the target municipalities, and the sector focus without needing to be reminded. This is the foundation that a more reliable evidence layer could be built on.

The SANTA architectural framework itself holds up. The concept of a Trust Discovery Engine that separates trust signals into verifiable dimensions, assigns confidence scores to data sources as well as to companies, explicitly represents uncertainty, and escalates to human validation when confidence falls below threshold — this is a coherent and valuable design. The architecture documents are a real contribution even if the data pipeline failed.

What we propose to fix this

The core proposal is simple: SANTA needs a data layer that is completely separated from the agent's generative context.

In practice this means treating the output database as an append-only evidence log rather than a generated document. Every entry in the log requires three fields before it can be written: the claim itself, the source it came from with a verifiable reference, and the retrieval timestamp. An entry without all three fields cannot be written. The agent cannot fill in missing fields by inference. If a KvK number cannot be retrieved from the KvK API or a confirmed web source, the field stays empty and is flagged as unverified rather than populated with a plausible-sounding fabrication.

This requires an intermediate persistence layer between the agent's working memory and the output files. Every time the agent retrieves a verifiable fact, it writes a raw evidence record immediately — a lightweight JSON object containing the claim, the source, and the confidence level. The final trust vector for a company is then computed from these raw evidence records, not from the agent's contextual reconstruction of what it found. If the raw

evidence records are sparse or absent, the trust vector reflects that honestly with low confidence scores and explicit gaps rather than high scores derived from nothing.

The second fix is a clear uncertainty halt condition. When the agent detects that it has run the same search query more than twice without new results, or that its source tools are returning errors, it should generate a gap report rather than a completion report. "I searched for KvK registration data for this company and could not confirm it. Confidence: 0.12. Recommended action: manual verification." This is more useful than a hallucinated KvK number with a 0.89 trust score.

The third fix is a multi-agent handoff model. The current architecture asks a single agent to do everything: search, retrieve, verify, score, and write. Under context pressure, the verification and writing steps degrade first because the search and retrieval steps are more immediately rewarded. Separating these responsibilities — a retrieval agent that writes only raw evidence records, a verification agent that cross-references those records, and a scoring agent that computes trust vectors from verified evidence — creates natural checkpoints where hallucination cannot propagate forward without being caught.

What we are asking for

We are not asking for a prize for something that did not work. We are asking for a conversation about what comes next.

The real question this sprint raised is whether OmegaClaw can be extended with an evidence provenance architecture that makes the boundary between retrieved facts and generated text explicit and enforceable. We believe it can. The memory infrastructure is already there. The agentic loop is already there. What is missing is a layer of epistemic discipline that treats uncertainty as information rather than as a gap to be filled.

SANTA is the right use case to build this on, because the consequences of hallucinated trust data are concrete and visible. A fake KvK number in a trust database does real harm. That makes it a good discipline for building the honesty mechanisms that beneficial AI needs at a more general level.

We would like to continue this work with the BGI community. The architecture documents are open source. The failure analysis is on the table. The next step is real.

Kristian Dansen · HandyHouseHelp · kristiandansen@gmail.com
github.com/KristianVASD/S.A.N.T.A. · MIT License